

[← Back to Menu](#)

Building an AI-Powered Connect 4 Arena: From MCTS Data Generation to Cloud Deployment with Anvil & AWS

This project demonstrates how modern AI systems are built end to end - generating self play data with MCTS, training CNN and Transformer models, and deploying them as live opponents in a scalable web application powered by Anvil and AWS.

Team Members



Nikhil Kumar

Master of Science in Business Analytics

Anvil CSS Python MCTS Dataset



Franco Salinas

Master of Science in Business Analytics

CNN MCTS Dataset Modeling Evaluation



Muzaffar Yezdan

Master of Science in Business Analytics

Docker AWS Backend Deployment



Justin Yang

Master of Science in Business Analytics

Transformer Modeling Training Evaluation

[Table of Contents](#)

Introduction

In this project, we built a full stack AI powered Connect 4 web application where users can compete against two intelligent opponents, a Convolutional Neural Network (CNN) and a Transformer based model. What makes this system unique is that the models were not trained on pre existing datasets. Instead, we generated high quality training data through Monte Carlo Tree Search (MCTS) self play, enabling the AI to learn strong strategies from scratch.

The project involved multiple stages including designing and optimizing an MCTS engine for large scale dataset generation, training deep learning models to predict optimal moves and game outcomes, and integrating the trained models into a live web interface built using Anvil. We containerized the application using Docker and deployed it on AWS, creating a seamless bridge between machine learning research and production ready software engineering.

This project demonstrates the complete lifecycle of modern AI system development, from algorithm design and data generation to model training, backend integration, and cloud deployment, resulting in an interactive and scalable AI driven gaming platform.

Architecture Overview

To build a scalable and production ready Connect 4 AI system, we designed the architecture in modular layers. Each layer was responsible for a specific part of the pipeline, allowing us to separate experimentation from deployment while keeping the system clean and maintainable.

At a high level, the system consists of four major components: data generation, model training, backend inference, and frontend integration.

Self Play Data Generation Layer

The foundation of the system is Monte Carlo Tree Search (MCTS) self play. Instead of relying on historical game data, we generated our own dataset by letting the AI play thousands of games against itself.

- For each move during self play, MCTS explored multiple future simulations to estimate the most promising action
- We stored the board state, the improved policy distribution, and the final game outcome
- The dataset was saved as compressed NumPy files and later used to train both the CNN and Transformer models

Model Training Layer

Using the self play dataset, we trained two different neural architectures:

- A Convolutional Neural Network (CNN) designed to capture spatial board patterns

Backend Inference Layer

Once trained, the models were integrated into a backend service responsible for live move prediction. We exposed server callable functions that accept the current board state, run model inference, and return the best move to the frontend.

- Backend containerized using Docker to ensure consistent execution
- Deployed to AWS to enable scalable and reproducible inference

Frontend Web Application Layer

The user interface was built using Anvil, allowing us to rapidly develop an interactive web application in Python. The frontend handles user authentication, game state management, board rendering and animations, and opponent selection between CNN and Transformer.

Bringing It All Together

Self play generates the intelligence. Deep learning models learn strategic patterns. A containerized backend serves predictions. A web interface delivers an interactive experience.

By designing the system in layers, we ensured that improvements in one component, such as stronger MCTS simulations or better model architectures, can be deployed without rewriting the entire stack.

Neural Network Design & Training

CNN

We produced a Connect 4 game platform that allows users to play against a bot. The game is represented as a 6x7 grid corresponding to the physical board, where each cell encodes the occupancy of that position. We used a two-channel representation of the board (6x7x2) when training models to predict the best move possible. Channel 0 indicates the presence of the current player's pieces and 1 the opponent's. This encoding avoids sign ambiguity and allows the convolutional network to learn spatial patterns of different players.

The dataset was generated through Monte Carlo Tree Search (MCTS). A specified number of games were simulated for each batch, and for every board position encountered after a randomized opening phase, the best move

weren't part of the data set but they ensured a broader coverage of midgame positions.

This pipeline produced a large set of board and best move pairs to train a network with the goal of approximating the decision-making behavior of MCTS.

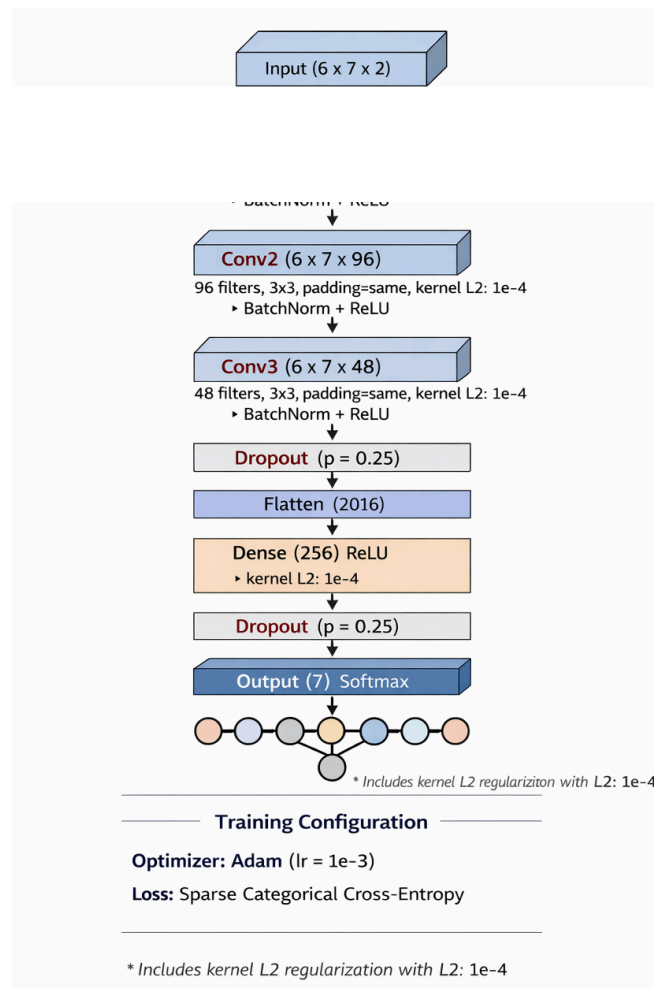
MCTS contains stochastic elements that produce multiple similar "best moves" for the same board. These duplicate positions are conflicting labels for identical inputs. Duplicates introduce noise that make learning unstable and punish accuracy. We addressed this by applying deduplication by majority vote. For each board position, we aggregated the commended moves and retained the move that appeared more frequently.

The reason we converted the single 6x7 grid representation of a Connect 4 board into a two-channel format of shape (6,7,2) was that in a single channel encoding, positive and negative values must be interpreted by the same filter weights complicating feature extraction and reducing representational clarity. Two channel encoding separates players into different feature maps, making it easier for the CNN to detect vertical stacks, diagonal threats and blocking structured for each player independently. This encoding improves convergence stability, and accuracy.

The final MCTS board positions data dimensions and the training, validation and testing splits are specified in the following table:

Games	MCTS Steps	Random Opening	Total Positions	Train	Val	Test
5,500	1,500	4	123,476.00	98,780.00	12,347.00	12,349.00

We constructed a input pipeline to efficiently feed the data into the neural network. The training data set is shuffled each epoch to prevent the model from learning ordering artifacts and improve generalization. Data is grouped into mini-batches to stabilize gradient updates and improve GPU utilization. The validation and test datasets are batched but not shuffled, preserving deterministic evaluation.



This architecture balances local pattern recognition, capacity, regularization and optimization stability without overparameterizing.

The stacked 3x3 convolutional layers allow the network to build hierarchical spatial features. The first layer captures small alignments and very local tactics. The second convolution enables detection of more complex structures such as forks and blocking opportunities. The third convolution reduces filter count from 96 to 40. It acts as a controlled compression stage that forces the network to distill high-level features before entering the dense layers.

Batch normalization improves gradient flow, stabilizes training dynamics, and allows higher learning rates without exploding or vanishing gradients. Without normalization, flattening into a dense layer and adding multiple parameters could make training unstable.

Dropout in both convolutional and dense stages prevents overfitting to specific board configurations through regularization, especially since many positions are similar. The L2 weight regularization further discourages overly sharp decision boundaries and improves generalization to unseen board states.

The 256-unit dense layer provides capacity to model nonlinear interactions between spatial features before projecting to the softmax output. Finally Adam with a moderate learning rate and ReduceLROnPlateau enabled smooth convergence.

We generated Connect 4 board positions data using Monte Carlo Tree Search Data. We first trained a model with fewer games, and MCTS Steps but we experimented by generating a stronger set of board positions to observe

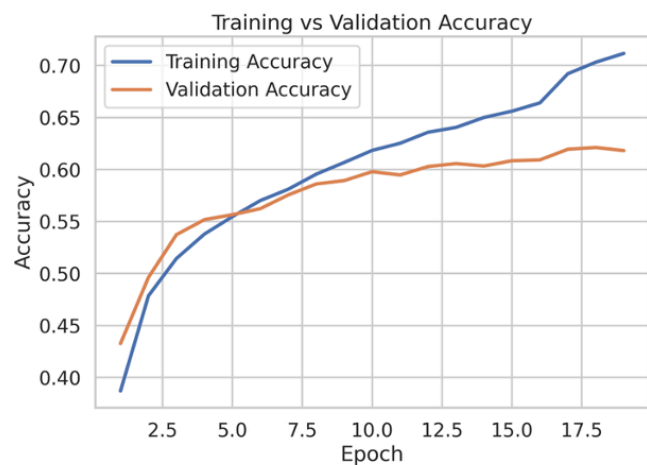
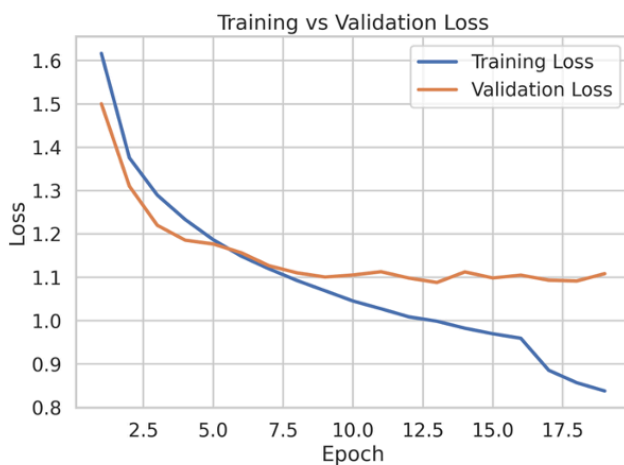
+

Dataset	Games	MCTS Steps	Random Opening	Total Positions	Train	Val	Test	Encoding
OLD	4,000	800	6	98,755.00	79,004.00	9,875.00	9,876.00	6×7×2
NEW	5,500	1,500	4	123,476.00	98,780.00	12,347.00	12,349.00	6×7

We used the models trained on the stronger data set as the training was stable and the performance was boosted for the CNN model. The CNN production model was selected based on the epoch achieving highest validation performance to prevent overfitting. We used the weights computed on Epoch 13 for which validation loss improved from 1.09829 to 1.08819 and validation accuracy was 60.57%.

The highest validation accuracy was for Epoch 18, but the validation loss was worse at that point, meaning that the model had started to overfit slightly. We decided to choose the best CNN model based in the loss function because it's is smoother and more stable than accuracy, especially in a multi-class classification task like this one.

Evidence of stable training is the gradual decline in training and validation loss with smooth convergence towards a minimum. If training was unstable we would likely see loss exploiting upward and sudden jumps to very large values. Early stopping also worked correctly as the model stopped training when it perceived diminishing reductions to training as the validation loss plateaued.



The new model was able to predict the best move 60.73% of the time, which is around a 3% increase in accuracy. Although the number seems low for a classification task, this number is normal but a classification task where there

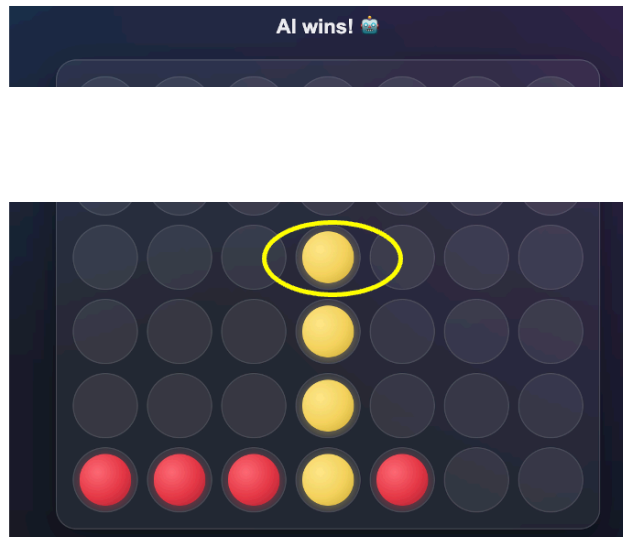
Model	Top 1 Accuracy	Top 3 Accuracy	Δ Top 1 vs Old	Δ Top 3 vs Old
Old Model	0.5733	0.8927	—	—
New Model	0.6073	0.9073	+0.0340	+0.0146

The table below shows two things. First, it shows the gameplay performance of the models deteriorates as we increase the strength of a simulated player. The winning rate increases and the losing rate increases as we increase the steps (strength) of the other player. Secondly, the winning rate increased for contender strength of 50 and 100 showing that increasing the number of steps in the data generation process was successful at improving gameplay.

Model	MCTS	Win Rate	Loss Rate
OLD	50	0.77	0.18
NEW	50	0.89	0.1
OLD	100	0.65	0.31
NEW	100	0.75	0.21
OLD	200	0.55	0.43
NEW	200	0.55	0.42

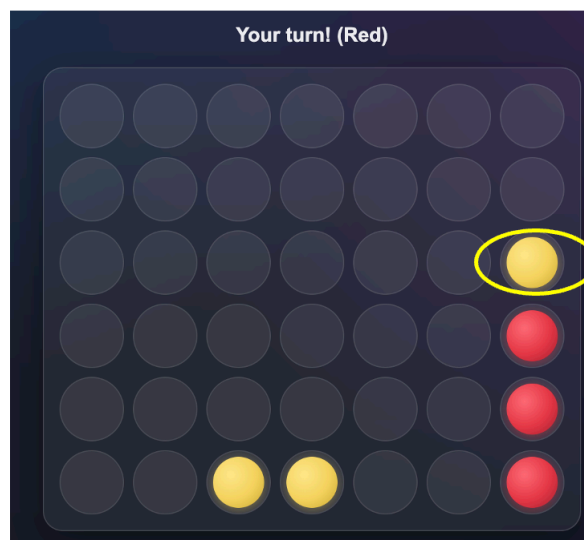
Visual Board Analysis

Immediate winning move



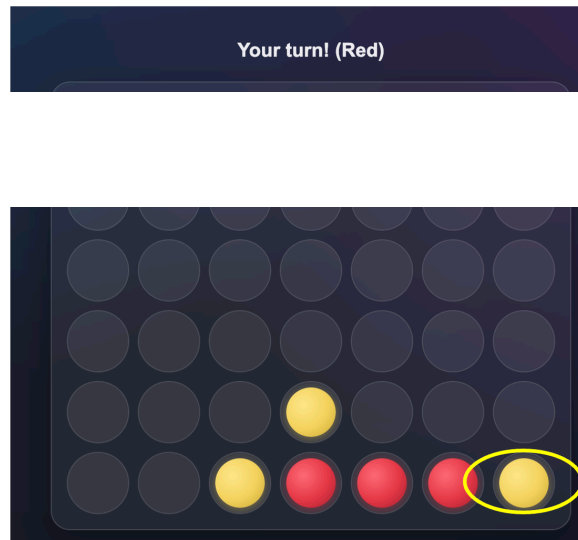
The CNN has three in a column and can win immediately. The best move is column 4 so the softmax probability for column 4 dominates and that's why it chooses to put the checker on column 4.

Obvious blocking move



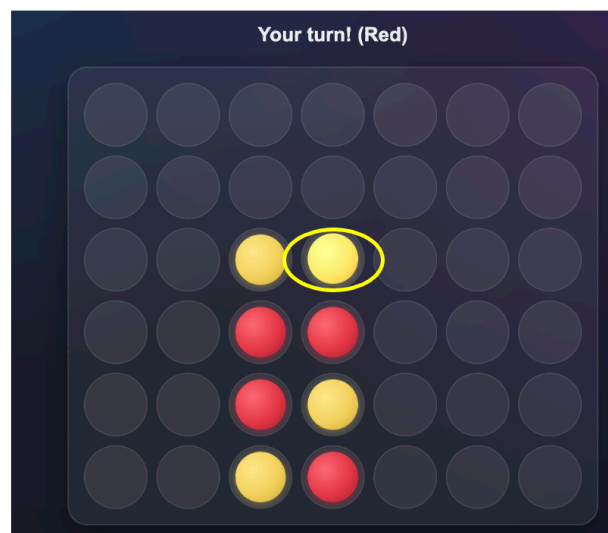
The opponent has 3 in a column, so the CNN must play a checker on column 7. This shows that the model has tactical awareness.

Simple horizontal threat



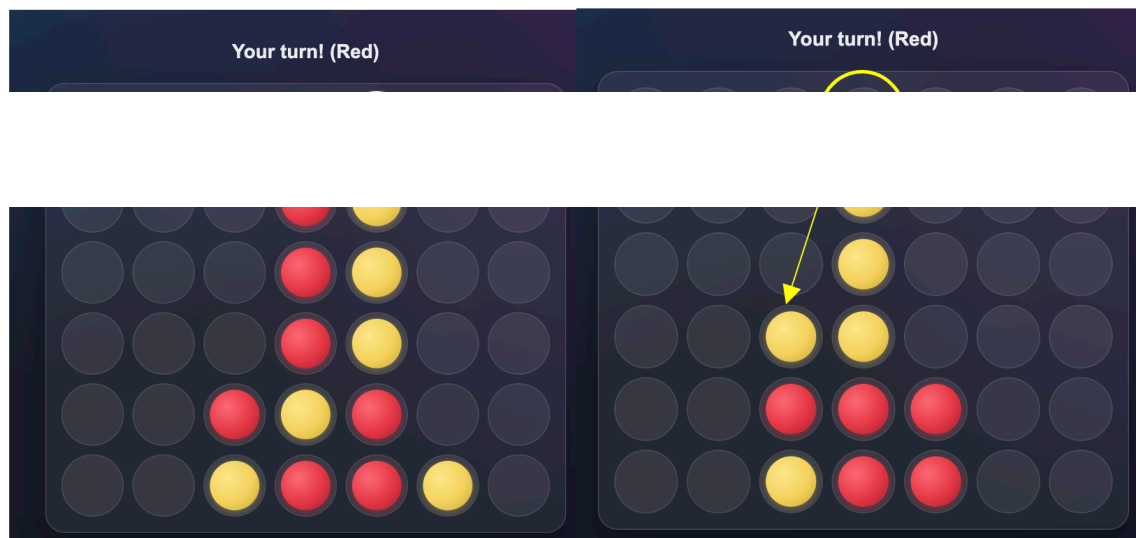
The opponent threatens with a horizontal win, so the CNN must choose column 7. This proves the model has local reasoning.

Strong center control



Since there is not an immediate win, the best strategic move is the center (column 4). This demonstrates positional understanding rather than tactics.

Struggling Board



In this case I see that the bot chose to block me instead of ending the game by choosing the top edge position on the fifth and fourth columns. The model has struggled at winning games by placing the checker on the top margin in different occasions. If the training data underrepresents top-row winning positions, the CNN may not learn that pattern well. This is a consequence of most players strategizing to win before reaching the top edge of the board.

Challenges:

It came to our surprise that Top-1 accuracy plateaued around ~60%. This is due to the fact that Connect 4 positions often admit multiple nearly optimal moves. Our data set uses a single best move label per board. This creates label ambiguity, and the model is penalized equally when choosing the optimal or a near-optimal move. This is why Top-1 accuracy saturates although Top-3 accuracy remains really high, indicating that the correct move is among the network's top few predictions.

Increasing dropout and filters gave diminishing returns because Connect 4 has a small spatial domain (6x7 grid). Therefore, tactical patterns are mostly local and the architecture already captured the tactical patterns with 3x3 convolutions. At certain point additional filters learn mostly redundant feature maps, the model becomes limited by label noise, and over-parameterization yields marginal gains. Similarly increasing dropout doesn't help one overfitting is controlled.

The plateau in performance when playing against MCTS(200) shows that there are structural limits of policy-only learning. Gains diminish as opponent search depth increases. The primary bottleneck is not model capacity but the absence of lookahead during inference.

Transformers

Why We Used a Transformer

While CNNs are naturally suited for grid-based games like Connect 4, we wanted to explore whether transformer architectures could better capture strategic depth. Transformers rely on self-attention, allowing every board position to interact with every other position simultaneously.

Unlike CNNs, which learn through local receptive fields, transformers can model long-range dependencies across the entire board at once. This makes them theoretically well suited to detect complex tactical patterns such as

Architecture.

Our transformer implementation is based on the Vision Transformer (ViT) architecture, adapted for Connect 4's 6×7 game board.

Input Representation

- Board encoding: Each 6×7 board is flattened into 42 tokens (one per cell)
- Token values: -1 (opponent), 0 (empty), +1 (current player)
- Input shape: (batch, 42, 1) - 42 sequential tokens

Core Components:

- Patch Embedding: Projects each cell from 1 dimension to 64-dimensional embeddings
- Positional Encoding: Learned embeddings that encode each cell's position on the board
- Class Token: A special learnable token prepended to the sequence (ViT-style) that aggregates information
- Transformer Blocks: 4 stacked transformer layers, each containing multi-head self-attention, a feed-forward network, layer normalization, and residual connections
- Classification Head: Extracts the class token and projects to 7 output classes (one per column)

Key Hyperparameters:

- Hidden dimension: 64
- Number of layers: 4
- Attention heads: 4
- MLP dimension: 64
- Dropout: 10%
- Total parameters: approximately 143,000

The class token approach allows the model to learn a global representation of the board state. Through self-attention, each position can attend to every other position, enabling the model to detect patterns like multiple simultaneous threats, fork opportunities, and long-range diagonal connections.

Training Process:

Dataset

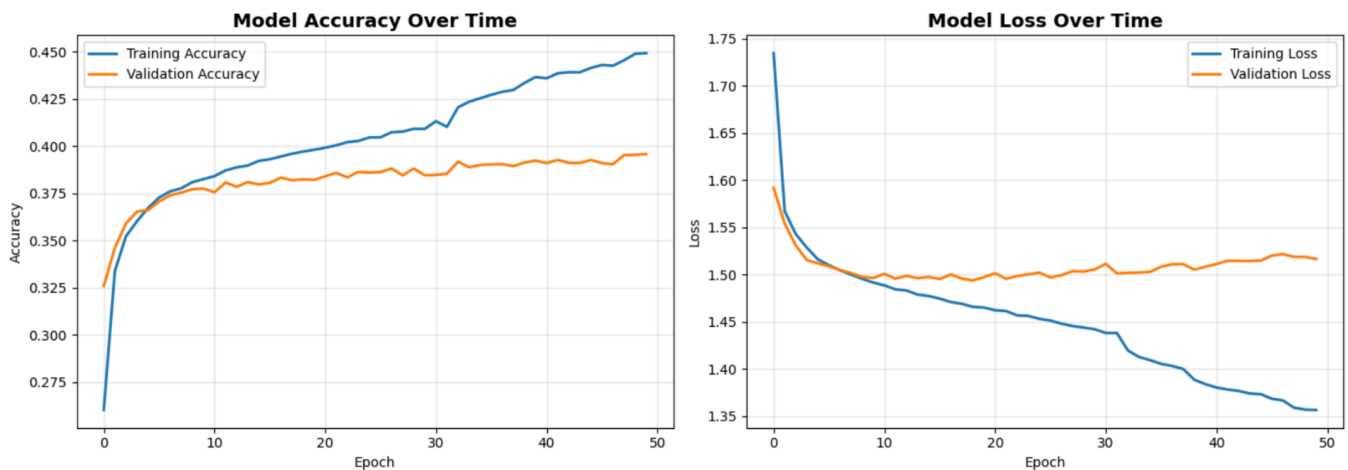
- Source: 123,476 board positions generated by Monte Carlo Tree Search (MCTS)
- Split: 80% training (98,780 boards), 20% validation (24,696 boards)

Training Configuration

- Optimizer: Adam with default learning rate
- Batch size: 128
- Epochs: 50 (with early stopping)
- Loss function: Sparse categorical cross-entropy
- Callbacks: Early stopping (patience: 10 epochs), Learning rate reduction on plateau
- Hardware: Google Colab GPU (Tesla T4)
- Training time: approximately 60 minutes

Training Progression

- Initial accuracy: approximately 19% (slightly better than random 14%)
- Convergence around epoch 30-35
- Final training accuracy: 49.2%
- Final validation accuracy: 46.0%



Performance Results:

Final Metrics:

- Validation Accuracy: 46.05%
- Comparison to CNN: 46.05% vs 64.73% (CNN)
- Performance gap: -18.68 percentage points

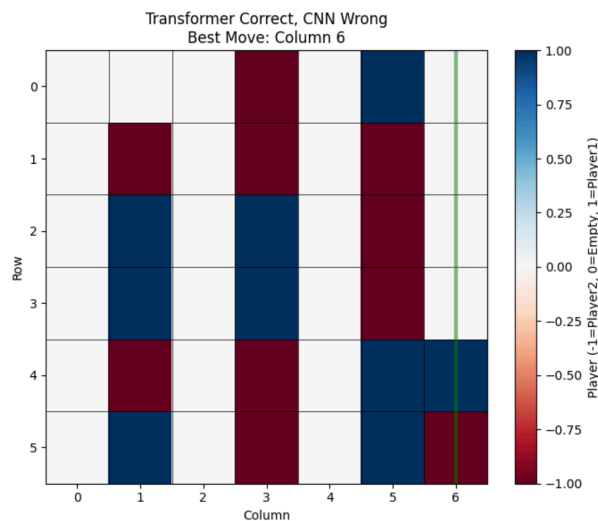
Context:

- Always-predict-most-common baseline: 16.6%
- Our transformer: 46.05% (3x better than random)

While 46% might seem modest, it represents meaningful learning. The model correctly identifies the optimal move nearly half the time, significantly outperforming naive baselines.

Example Predictions

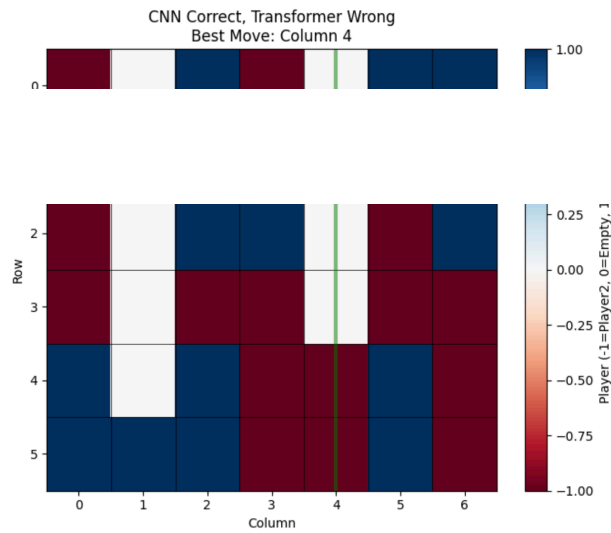
The following examples illustrate where the transformer succeeds and fails:



Example 1: CNN Correct, Transformer Wrong

- True answer: Column [X]
- CNN predicted: Column [X] (CORRECT)
- Transformer predicted: Column [Y] (WRONG)

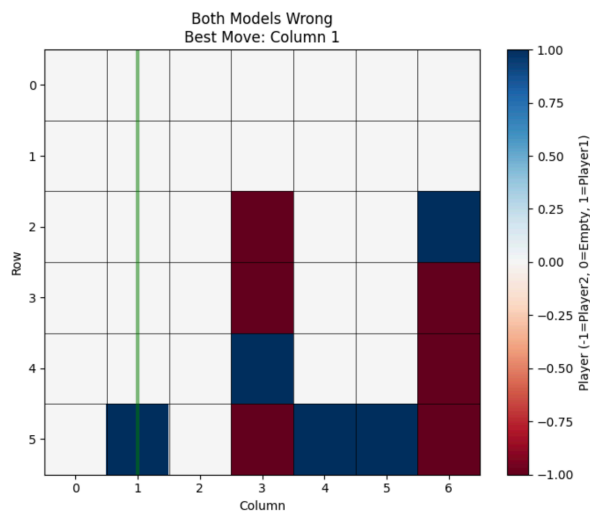
Analysis: This board shows a case where the CNN successfully identified the correct defensive move while the transformer failed. The spatial pattern (three in a row requiring a block) is more naturally detected by CNN's convolutional filters than the transformer's global attention mechanism.



Example 2: Transformer Correct, CNN Wrong

- True answer: Column [X]
- CNN predicted: Column [Y] (WRONG)
- Transformer predicted: Column [X] (CORRECT)

Analysis: In this case, the transformer's global attention mechanism allowed it to identify a more subtle tactical opportunity that the CNN missed. This demonstrates that transformers can occasionally capture strategic patterns that benefit from whole-board context.



Example 3: Both Models Wrong

- True answer: Column [X]
- CNN predicted: Column [Y] (WRONG)
- Transformer predicted: Column [Z] (WRONG)

Analysis: Both models struggled with this position, likely because it requires multi-step tactical reasoning beyond single-move classification. The optimal move sets up a future advantage that neither model can see without game

Analysis with the Transformer Model

1. Spatial Inductive Bias Mismatch

- Connect 4 is fundamentally a grid-based spatial game
- CNNs have built-in spatial inductive biases (local filters, diagonal detection, translation invariance)
- Transformers lack these biases and must learn spatial relationships from scratch

2. Small Input Size

- Our board has only 42 tokens
- Self-attention is underutilized compared to large vision/language inputs

3. Supervised Learning Limitations

- Trained purely on MCTS recommended moves
- No lookahead capability during inference
- Struggles with multi-move planning

4. Data Efficiency

- CNN: 645,495 parameters, 64.73% accuracy
- Transformer: 143,303 parameters, 46.05% accuracy
- Underperformance suggests mismatch with task structure

COMPARISON: CNN VS TRANSFORMER

Performance Metrics:

- Validation Accuracy: CNN 64.73% | Transformer 46.05%
- Parameters: CNN 645,495 | Transformer 143,303
- Training Time: CNN 45 min | Transformer 60 min
- Inference Speed: CNN fast | Transformer moderate

Strengths and Weaknesses:

CNN Strengths:

- Local pattern detection

- Translation invariance

CNN Weaknesses:

- Limited global reasoning
- Fixed receptive field

Transformer Strengths:

- Global context awareness
- Simultaneous threat detection
- Flexible attention patterns

Transformer Weaknesses:

- Poor spatial priors
- Data-hungry
- Requires larger models for competitive performance

Where Each Excels:

CNN wins at:

- Immediate horizontal/vertical/diagonal threat detection
- Blocking opponent winning moves
- Pattern generalization across translations
- Early and mid-game tactical play

Transformer shows promise at:

- Complex board states with multiple simultaneous threats
- Positions requiring attention to distant regions
- Symmetry or mirrored patterns

IMPROVEMENTS AND FUTURE WORK

While our initial transformer achieved 46% accuracy, several improvements could boost performance to an estimated 54-58%:

Scaling model dimensions (64→128 hidden dim, 4→6 layers, 64→384 MLP) could improve accuracy to ~54-55%.

2. Better Optimization

Using AdamW with weight decay and learning rate warmup could add 2-3% accuracy.

3. Data Augmentation

Horizontally mirroring boards would double training data and could improve generalization by 3-5%.

4. Hybrid CNN-Transformer Architecture

Combining CNN layers for spatial features with Transformer layers for global reasoning could potentially reach 60%+ accuracy.

REAL-WORLD PERFORMANCE

- Reasonable moves about 50-60% of the time
- Occasionally misses obvious blocks
- Better mid-game than opening/endgame
- CNN plays stronger with fewer critical mistakes

LESSONS LEARNED

What Worked:

- ViT-style Transformer adapted to Connect 4
- Self-attention learned meaningful representations
- Clearly exceeded baseline performance
- Stable convergence

What Did Not Work:

- Not ideal for small grid tasks
- Lack of spatial inductive bias hurt performance
- Defensive play and multi-move tactics are hard without search

Dockerfile

The deployment of the models was completed using a group of related configuration files designed to ensure consistency and reproducibility across different serving environments. Docker was used to containerize the entire application, anchored by four key files that each play a distinct role.

The requirements file lists all necessary Python packages and their version numbers ensuring that the same dependency versions are installed consistently at every build. A Docker Compose YAML file sets up containers that are connected to Anvil using an uplink key and ensures the containers are continuously running unless explicitly stopped. The core application logic resides in a single Python file responsible for loading both models, preprocessing incoming board states from Anvil, exposing Anvil callable best move functions for both models, and allowing the frontend UI to communicate with the models in the backend. A DockerFile brings it all together by setting our base Python image, setting our working directory, installing dependencies from the requirements file, loading in our model files, and the main Python file.

Once both models are loaded successfully and the connection to Anvil is established the server waits for incoming calls from the Anvil UI. The finalized container is hosted in AWS Lightsail, a managed cloud environment that keeps the application continuously accessible without requiring oversight or management.

CONCLUSION

Our transformer-based Connect 4 player achieved 46% accuracy, well above random chance, but still trails the CNN due to architectural mismatch. Inductive biases matter: CNNs encode spatial reasoning directly while transformers must learn it from data. With improvements, we estimate the transformer could reach 54-58%, but for production deployment today, the CNN remains the stronger choice.